

OPS — the live bot and the store

Owens: engine/mag_bot.js , engine/showdown_bot.js , the Showdown connection, OTS ingest, replay publication, the team pool, data/live-games/ .

Its one number: usable share of the store, and battles recorded.

May not: block any other division. This is the one place where an interruption costs a real game rather than a re-run.

<!-- GENERATED: engine/status.js -->

```
OPS — the live bot and the store
store: 88091 games, 29934 usable (34%), 25991 teams (live.js 2026-09-06)
live-games/: 34 battles recorded
data/games.ladder.jsonl      last written 2026-09-04 01:30
data/games.bo3.jsonl        last written 2026-09-04 01:30 <- the Force-OTS format, collected hourly
data/games.ots.jsonl        last written 2026-08-21 22:35 <- FROZEN external import, complete; date is
```

stamped 2026-09-06 11:57

<!-- /GENERATED -->

TWO STORE DEFECTS THIS LEDGER HELD AS PROSE ARE NOW REGISTER ROWS, AND NEITHER HAS AN INSTRUMENT. 2026-09-04, CHANGELOG 5.246.0

NOTHING IN THE STORE MOVED IN THIS PASS. No ingest, no bot session, no publication, no re-parse. What changed is where two of this ledger's own findings live. The 5.244.0 section below says of the data/games.selfplay.jsonl duplicate ids and of the raw-log census artifact that they were "*named so it cannot look closed*" — and at the time they were named only in this document. They are now filed as ROADMAP #536 and #540, both INSTRUMENT OWED : nothing in the tree decides either, and the rows say so rather than citing a probe that does not exist. That section stands as written and is superseded from here rather than rewritten.

THE POINT IS THE MECHANISM, NOT THE BOOKKEEPING. A defect recorded only in a dated ledger section is a defect nobody can enumerate, which is the same failure as the fourteen typed handoffs. Six such defects were filed this pass (register 497 → 503 rows, 251 → 257 open); five of the six carry no VERIFIED BY , which is the honest state of them.

node engine/status.js --write **is still OWED**, so the <!-- GENERATED --> block above still carries its 2026-09-01 store figures and must not be read as today's.

AN OUTSIDE USAGE TABLE IS ARCHIVED AS A COMPARISON SET THAT FEEDS NOTHING, AND THE STORE'S CONTAMINATION IS

RE-MEASURED RATHER THAN RE-DISCOVERED — 2026-09-04, CHANGELOG 5.245.0

THE INGEST ALREADY EXISTED AND ITS CRON WAS DUE TO FIRE TODAY, SO IT WAS RUN RATHER THAN DUPLICATED. `engine/fetch_smogon_stats.js` pulled Smogon's August 2026 statistics: **310 species, 1,269,250 bo1 battles, zero illegal entries.** The Champions SP fingerprint was checked rather than assumed — **66/32, zero violations** — because the header carries no format name and a filename proves nothing.

IT IS A COMPARISON SET. IT FEEDS NOTHING. No model, no prior, no filter and no view reads it. It exists to be diffed against our own store, which is the only thing an outside table is safe for here: it is a different population collected under different rules, and the moment it feeds a decision it becomes an unstamped input to everything downstream.

OUR STORE HOLDS 85 SPECIES THAT ARE `isNonstandard: 'Past'` — the known contamination, measured again rather than recalled. **Species Smogon has seen that we never have: 0.**

AND THE FIRST JOIN WAS WRONG IN A WAY THAT LOOKED LIKE A FINDING. A naive join said 71, including Charizard-Mega-Y at rank 6 with **26.52%** usage — a species we obviously do see. `durable-ingest.js` collapses megas to their base forme in `six`, so the seen-set has to be `six ∪ sets`; that was controlled both ways before the corrected number was written down. **bo1 and bo3 are different metagames and the same species proves it: 26.52% in one, 50.46% in the other.** Nothing that pools the two stores is measuring one population.

WHAT THIS DIVISION DID NOT DO. No store was rewritten, no re-parse was run, no live game was played, and `data/meta-usage.json` was not regenerated. The store figures in the `<!-- GENERATED -->` block above are from the last `status.js --write` and are one pass behind; that run is OWED.

THE RAW LOG IS THE SOURCE OF TRUTH AND THE STORE IS A DERIVED VIEW — 2026-09-04, CHANGELOG 5.244.0

THE INVERSION BEHIND A DOZEN STORE REWORKS. The REGENERABLE artifact was durable and the IRREPLACEABLE one was gitignored. Everything below follows from putting that the right way round: the store can be rebuilt from the raw log; nothing can rebuild the raw log.

- **S1 — the filter ran before the archive write.** `engine/durable-ingest.js:543` continued on the completeness check BEFORE the log was archived, so a game the CURRENT parser could not read had its raw log **deleted** — destroying the one artifact a FUTURE parser could have used. That is the invariant broken at its source, not downstream of it.
- **S2 — the row was written before the log,** on independent streams, so a crash left an **orphan row**: the wrong direction. Two passes now, through one exported `archiveThenStore()`, with store output verified byte-identical.
- **S3 — the archive is now write-once dated shards,** not one blob. Caught mid-test before it shipped: a `-2` collision suffix sorts BEFORE `.jsonl`, which replays an append-only archive out of order.
- **S4 — `get()` resolved an empty string on HTTP error, on timeout AND on an empty body.** Three facts, one value, which is why a dead API and a quiet day were indistinguishable. It returns null on failure now, with the discriminator derived from the endpoint rather than assumed.

A PUBLISHED ESTIMATE IS REVERSED. The single-blob archive was described as roughly **65 days** from trouble. Measured, compression is **13.98%** and a single blob is **78 MB across both archives today** — **already 78% of the 100 MB limit.** The estimate was wrong; the shard design is what the measurement

implies, not a preference.

AND THE GUARD THAT PROTECTS A REBUILD ASSUMED THE OLD EQUALITY.

`engine/rebuild_records.js:117` compared COUNTS. Now that the archive is a SUPERSET by design it would have refused every valid rebuild — **and it was already letting a bad one through**: with a truncated log the counts balanced `3 == 3` and it swapped in a corrupted store at exit 0, while another arm lost a game outright. It now filters on completeness and re-asks the guard **by id**, naming any game that would be lost. Strictly stronger than what it replaced.

RECOVERED: 7,275 raw logs across both stores — 6,661 ladder, 614 bo3, 0 unavailable, 0 without a timestamp. Both archives are complete supersets, `MODE=reparse` is unblocked, and the store-orphan section of `engine/sweep.js` is clean. **This is the first time a re-parse is safe to run**, which is the whole point of the ordering fixes above: a re-parse under S1 would have deleted every log the current parser cannot read.

THE SHAPE ALL OF THIS ARRIVED IN. `engine/durable-ingest.js:464` already explained the archive drift in a comment, and that comment already said *"RUN THIS BEFORE ANY REPARSE"*. Nothing was unknown; nothing read it. That is this session's single finding, and OPS is where it cost the most, because the thing at risk was the only artifact in the project that cannot be regenerated.

NOT FIXED, ROUTED, AND NAMED SO IT CANNOT LOOK CLOSED. `data/games.selfplay.json1` holds **89 duplicate ids**. The raw-log census artifact still asserts the old subset relation and has no generator to correct it. Neither was touched.

WHAT WAS NOT RUN. No ingest, no bot session and no publication was performed by this pass, and `node engine/status.js --write` was not run — so the `<!-- GENERATED -->` block above still carries its 2026-09-01 store figures and must not be read as today's. Full accounts: eleven reports under `docs/_reports/2026-09-04-*.md`.

Standing state

MAGABRA is **locked** on Showdown (`!magabra`): it can battle and save replays, it **cannot chat**. `--trash` is wired and silently dropped by the server — that is the lock, not a code bug.

`SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown` is required for anything that touches the simulator.

The password is in `data/.showdown-pass` (gitignored). **Never type it into a command.**

The rules that are not negotiable

- **NEVER restart the live bot mid-battle.** It forfeits Will's game. There is no urgency that outranks this.
- **OTS only.** Non-OTS games are thrown out; OTS games are recorded and the replay auto-published.
- **Never** `git add -A` **or** `git add -u`. The ingest churns `archetypes.json`, `kad-replays.js`, `live.js` and `conformance.json` on every run, so a broad add sweeps generated noise into the commit. Add files by name.
- **6 processes maximum**, and ask before running wide while Will is at the keyboard. RAM is the real cap, not cores.

Store contamination — measured 2026-08-27, NOT acted on

Full account: `docs/_reports/2026-08-27-contamination-refresh.md`.

`data/store-validation.json` was refreshed against the current ladder store (**67,384 games**, 591,457 revealed sets); the previous reading was 2026-08-07 over 47,210. **The headline is an UPPER BOUND and must not be used as a filter key** — it splits into two unlike things:

class	games	rate	what it is
headline (any illegal set)	1,252	1.858%	upper bound, do not filter on this
species	76	0.113%	contamination — a team from another regulation
move-only	1,175	1.744%	mostly the Illusion signature ; 1,020 of them have a Zoroark line on the same side

A second, independent walk — every key in `sets / six / brought / lead` against `Dex.forFormat(...)` — returns **the same 76 games**. Two rulers, one number.

No filter has been added and no artifact was changed. The filter is Will's decision; this is the costing for it.

Reading the record

The head-to-head against Will sits around 15-4. **Do not read that as strength.** He plays unfamiliar teams deliberately. It is a measurement of his experiment, not of the bot.

Done looks like

- Usable share of the store trending up, not just total games.
- Every OTS game recorded with its replay published.
- Zero mid-battle restarts.

Backlog

- The team pool's base filter is completeness, not quality — that item is owned by [SEARCH.md](#) because it changes what gets measured, not how the bot runs.
- `~~ data/games.ots.jsonl` has not been written since July. Confirm whether OTS ingest is still landing in it or has moved to the ladder store.~~ **ANSWERED AND STRUCK 2026-08-27.** The code answers it, and the generated block above already said so — this line contradicted its own page.
`engine/ingest_ots.js` is a **manual importer**: it takes `logs_*.json` files as arguments and prints a usage line with none. It appears **nowhere in** `.github/workflows/` (`grep -rn ingest_ots .github/` returns nothing), so no scheduled run has ever written it and none was missed. It also **refuses** `--out data/games.ladder.jsonl` by name, so the ingest cannot have "moved to the ladder store" — the two corpora are different information AND incentive regimes and the importer will not pool them. The premise was stale as well: the file was last written **2026-08-21 22:35**, not July.